π-Former: Succinct Proofs of Transformer Inference via a SNARK-Friendly Architecture

Hideaki Takahashi
Columbia University
ht2673@columbia.edu

Abstract

Verifiable transformer inference lets an untrusted operator prove that a posted output was produced by a committed model, without revealing the weights. The obstacle is that a transformer block is built from exactly the operations that finite-field SNARKs find expensive: softmax, GeLU, LayerNorm, and dense projections. Prior systems leave the model untouched and pay this cost on every block.

We present π-Former, a succinct argument whose central design choice is to co-design the *model* so the proof system has nothing expensive to absorb. Three architectural changes drive the design: (i) softmax attention is replaced by a kernel attention whose feature map is realised as an additively decomposed learnable lookup, so the only non-arithmetic operation is a structured Lasso table look-up; (ii) every projection is constrained to ternary entries scaled by a per-layer scalar, eliminating all field multiplications from weight contractions; and (iii) LayerNorm is reformulated as an integer protocol provable through sumcheck and chunked range proofs. We build a transparent BN254 SNARK on top with Hyrax commitments, batching every per-block sub-protocol into a constant number of model-level sumchecks, a globally shared range-multiplicity commitment, and a zerocheck-folded inverse polynomial that removes $B{\cdot}C$ Hyrax commitments per range bucket. Residual connections require no proof: they are verified by linearity of the commitment scheme. We prove end-to-end soundness under discrete log on BN254 G1 and implement a complete prototype in ∼19k lines of Rust with a bit-identical PyTorch training pipeline.

1 Introduction

Large language models are increasingly deployed as black-box services. The user submits a prompt, an opaque operator runs a proprietary model, and the user has no cryptographic guarantee that the returned output came from the advertised model. Verifiable inference [1, 10, 12, 15] closes this gap with a succinct non-interactive argument: the operator commits to the weights once, and for every input produces a small proof that the verifier checks without re-executing the model and without learning the weights.

The core difficulty is that a transformer block is built from operations that finite-field SNARKs find expensive. Softmax requires exponentials and a row-wise division; GeLU is transcendental; LayerNorm contains a square root and a division by a running variance; a dense projection costs one field multiplication per weight entry. Prior work attacks this on the proof-system side: zkLLM [15] introduces `tlookup`, a parallelised lookup argument tailored to softmax and SwiGLU; zkGPT [12] pairs Plonky2 with constraint fusion to drive down the cost of approximating softmax. Both keep the model fixed and pay an "unfriendly-operation tax" on every block.

Our approach. π-Former starts from a different premise: change the *model* so that the proof system has nothing expensive to absorb. Three architectural principles drive the design:

- **SNARK-friendly operators.** Replace every operation without a compact arithmetic representation. Softmax becomes a kernel attention whose only non-arithmetic component is a structured table lookup; LayerNorm becomes an integer protocol provable by sumcheck plus chunked range proofs.
- **Ternary projections for free contractions.** Constrain every projection matrix to entries $\{-1, 0, +1\}$ scaled by a per-layer scalar α [11]. A degree-2 sumcheck against a ternary weight folds the equality polynomial against ternary entries with $+=$/$-=$/skip; no field multiplications appear in the contraction.
- **Learnable lookup tables.** The element-wise non-linearity ϕ is itself learnable: it is parametrised as a sum of small sub-tables and trained end-to-end. Under Lasso [14], prover cost depends only on table size and query count, not on table contents, so training adapts table values at zero proving cost.

Together these reduce a transformer block to operations that map directly to a low-degree sumcheck or to a Lasso lookup over a small structured table.

Model-level batching. On top of this architecture we build a proof system around a single observation: the same protocol runs once per block with independent inputs but identical structure. π-Former runs six model-level sumchecks (one per protocol type) that fold all L per-block claims via a Fiat–Shamir random linear combination; the inner sumcheck transcript is shared across blocks. The same idea applies to Lasso (one global multi-instance proof per lookup family), to range checks (one shared multiplicity commitment per width bucket spanning the entire model), to residuals (homomorphic Hyrax addition; no proof at all), and to the final opening stage (deferred batch accumulators finalised in one MSM pair per parameter group). For the range proof specifically, we replace the explicit Hyrax commitment to each LogUp inverse polynomial by a zerocheck folded into the bucket sumcheck; this removes $B \cdot C$ commitments per bucket without changing the per-round prover cost.

Contributions.

1. A SNARK-friendly transformer block (§3) in which attention, normalisation, and projection each map to a sumcheck or a Lasso lookup, combining learnable lookup attention, ternary projections, and a constraint-fused integer LayerNorm.
2. A complete protocol (§4) batching all L per-block subproofs into six model-level cross-block sumchecks, one global Lasso for the ϕ family, and a bucket-batched range proof shared across the whole model.
3. A zerocheck-folded LogUp range proof (§3.5, Theorem 5) that eliminates $B \cdot C$ Hyrax commitments per width bucket while keeping the bucket sumcheck cubic and soundness within $O(n/|\mathbb{F}|)$.
4. End-to-end soundness (§5, Theorem 8) reducing to discrete log on BN254 G1, and a working prototype in ∼19k Rust LoC with a bit-identical PyTorch training pipeline (§6).

Threat model. We target the standard non-interactive argument-of-knowledge setting in the random-oracle model. The verifier holds a verifying key vk binding the weights and lookup tables, a public input $\mathbf{x}_{\text{in}}$, and a claimed output $\mathbf{y}$, and accepts iff $\mathcal{M}_\theta(\mathbf{x}_{\text{in}}) = \mathbf{y}$ for the committed model. Soundness is computational under discrete log on BN254 G1, the standard assumption for the Hyrax PCS [18]. The current implementation is not zero-knowledge: weight commitments are public and intermediate sumcheck evaluations are revealed. The standard masking extension is discussed in §7.

2 Preliminaries

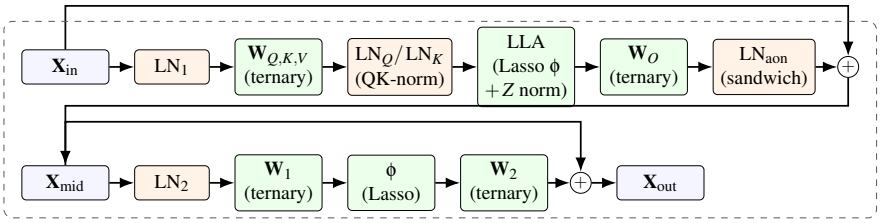
Notation. We work over the BN254 scalar field $\mathbb{F}$ ($r \approx 2^{254}$) and the BN254 G1 group $\mathbb{G}$. We use L for the number of blocks, T for the sequence length, d for the embedding dimension, and d_{ff} for the FFN width; the prototype is single-head ($d_h = d$). A multilinear polynomial $\widetilde{f}$ on n variables is identified with its 2^n evaluations on $\{0,1\}^n$, and the equality polynomial $\widetilde{\text{eq}}(r,x) = \prod_i (r_i x_i + (1-r_i)(1-x_i))$ satisfies $\sum_x \widetilde{\text{eq}}(r,x)\widetilde{f}(x) = \widetilde{f}(r)$. Real-valued tensors are quantised to integers and embedded in $\mathbb{F}$ before commitment; our training pipeline runs the same integer arithmetic, so the exported witness matches the field circuit bit-for-bit.

Transformer block. A pre-norm block [17] computes $\mathbf{X}_{\text{mid}} = \mathbf{X}_{\text{in}} + \text{Att}(\text{LN}(\mathbf{X}_{\text{in}}))$ and $\mathbf{X}_{\text{out}} = \mathbf{X}_{\text{mid}} + \text{FFN}(\text{LN}(\mathbf{X}_{\text{mid}}))$, with softmax attention $\text{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d})\mathbf{V}$ and LayerNorm [3] $\gamma \odot (\mathbf{x}-\mu)/\sqrt{\sigma^2+\varepsilon} + \beta$. The non-arithmetic primitives (exp, division, square root) dominate SNARK complexity in prior work.

Sumcheck. The sumcheck protocol [13,16] reduces a claim $H = \sum_{x \in \{0,1\}^n} f(x)$ for an MLE f of individual degree δ to a single evaluation $f(r)$ at random r, with soundness error $\leq n\delta/|\mathbb{F}|$ and verifier cost $O(n\delta)$. π-Former uses the degree-2 variant for products of two MLEs, the cubic variant for LayerNorm variance and our zerocheck-folded range proof, and a multi-batched variant $\sum_k \lambda_k \sum_x f_k(x) g_k(x)$ for cross-block batching.

Hyrax PCS. Hyrax [18] arranges $2^n = 2^{\nu+\sigma}$ MLE evaluations as a $2^\nu \times 2^\sigma$ matrix and commits to each row via a vector Pedersen commitment over transparent generators. The scheme is computationally binding under DL on $\mathbb{G}$, requires no trusted setup, and gives $O(\sqrt{N})$ commitments, openings, and proof size. Crucially, the row-Pedersen structure is linear: $\mathsf{Com}(\mathbf{A}) + \mathsf{Com}(\mathbf{B}) = \mathsf{Com}(\mathbf{A}+\mathbf{B})$, which π-Former exploits to verify residual connections without any proof.

Lasso. Lasso [2,5,14] proves that N queries are consistent with a public sub-table $T \in \mathbb{F}^{2^m}$ via one batched sumcheck. A composite table that decomposes *additively* into c sub-tables (Lasso's structured condition) is committed as c small tables of size $2^{B/c}$. Lasso's defining property for our design is that prover and verifier cost depend on table size and query count, but are independent of table *contents*: training the entries of T has zero proving-cost impact.



Cross-block batching. The five LNs per block (LN_1, QK-norm LN_Q/LN_K, sandwich LN_{aon}, LN_2), plus the final LN, are folded into one *model-level* batched LN proof. Ternary projections fold into `qkv`/`oproj`/`ffn` sumchecks; LLA inner products fold into `batch_attn_out`/`ctx`; the row-normaliser Z is enforced by a cubic multi-batched sumcheck with range proofs on the floor-division residuals (§3.1); Lasso ϕ folds into one global multi-instance proof per lookup family; range checks fold into one shared multiplicity commitment per width bucket; residuals are homomorphic Hyrax addition.

Figure 1: A single π-Former block. Boxes are matrices; rounded green nodes are operations the SNARK proves; orange nodes are LayerNorms. *Sandwich norm* (LN_{aon}) is applied to the attention output before the residual; *QK-norm* (LN_Q, LN_K) is applied to $\mathbf{Q},\mathbf{K}$ after the ternary projection and before ϕ. Ternary projections contain no field multiplications in the contraction; Lasso lookup cost is independent of table contents; residual $+$ is checked by homomorphic addition of Hyrax commitments and requires no proof.

3 The π-Former Architecture

A π-Former block (Figure 1) is a sandwich-normalised linear-attention block. With $\mathbf{X}_{\text{in}} \in \mathbb{F}^{T\times d}$, the block computes

$$
\begin{aligned}
\mathbf{X}_{\text{n1}} &= \text{LN}_1(\mathbf{X}_{\text{in}}),\\
\mathbf{Q},\mathbf{K},\mathbf{V} &= \mathbf{X}_{\text{n1}}\mathbf{W}_{Q,K,V} \ \text{(ternary)},\\
\mathbf{Q}^{\text{n}},\mathbf{K}^{\text{n}} &= \text{LN}_Q(\mathbf{Q}),\ \text{LN}_K(\mathbf{K}),\\
\mathbf{N} &= \phi(\mathbf{Q}^{\text{n}})\big(\phi(\mathbf{K}^{\text{n}})^\top\mathbf{V}\big),\\
Z_t &= \phi(\mathbf{q}^{\text{n}}_t)^\top\textstyle\sum_s\phi(\mathbf{k}^{\text{n}}_s),\\
\mathbf{Y}^{\text{att}}_{tj} &= \lfloor S\cdot\mathbf{N}_{tj}/Z_t\rfloor,\\
\mathbf{X}_{\text{mid}} &= \mathbf{X}_{\text{in}} + \text{LN}_{\text{aon}}(\mathbf{Y}^{\text{att}}\mathbf{W}_O + \mathbf{b}_O),\\
\mathbf{X}_{\text{out}} &= \mathbf{X}_{\text{mid}} + \phi(\text{LN}_2(\mathbf{X}_{\text{mid}})\mathbf{W}_1)\mathbf{W}_2.
\end{aligned}
$$

After L such blocks the model applies a final LayerNorm and a ternary LM head; the model contains $5L+1$ LayerNorms in total. The scale S re-introduces fractional precision before the integer floor division.

Two of the five LayerNorms per block are not in a textbook transformer but are essential for our integer pipeline. *QK-norm* (LN_Q,LN_K) pins the range of $\mathbf{Q},\mathbf{K}$ so that $\phi(\mathbf{Q}^{\text{n}}),\phi(\mathbf{K}^{\text{n}})$ and Z stay in the lookup-table range under any per-layer ternary scale. *Sandwich norm* (LN_{aon}) absorbs the unbounded magnitude of linear attention before the residual: softmax is row-stochastic and naturally bounded, but $\phi(\mathbf{Q})(\phi(\mathbf{K})^\top\mathbf{V})$ is not, and an unnormalised residual destabilises downstream LayerNorms. Both share one LN sub-protocol (§3.4); their only cost is extra LN witnesses, folded into the model-level batched LN proof and the bucketed range batch.

3.1 Learnable Lookup Attention

Softmax attention requires T^2 exponentials and T row-wise divisions. We replace the kernel with a positive-definite inner product $\kappa(\mathbf{q},\mathbf{k}) = \langle\phi(\mathbf{q}),\phi(\mathbf{k})\rangle$ on the post-QK-norm tensors, where $\phi:\mathbb{R}^{d_h}\to\mathbb{R}^{d_h}$ is an element-wise structured lookup (§3.2). The block computes the numerator $\mathbf{N}_{tj} = \phi(\mathbf{q}^{\text{n}}_t)^\top(\sum_s\phi(\mathbf{k}^{\text{n}}_s)\mathbf{v}_s^\top)_j$ and the per-row denominator $Z_t = \phi(\mathbf{q}^{\text{n}}_t)^\top\sum_s\phi(\mathbf{k}^{\text{n}}_s)$. The context matrix $\mathbf{M}=\sum_s\phi(\mathbf{k}^{\text{n}}_s)\mathbf{v}_s^\top\in\mathbb{F}^{d_h\times d_h}$ is computed once per block, giving $O(Td_h^2)$ prover cost instead of $O(T^2d_h)$ [7].

Floor-division proof for row normalisation. π-Former proves $\mathbf{Y}^{\text{att}}_{tj} = \lfloor S\mathbf{N}_{tj}/Z_t\rfloor$ in-circuit. The witness includes auxiliary tensors rem and diff with $\text{rem}_{tj} = S\mathbf{N}_{tj} - Z_t\mathbf{Y}^{\text{att}}_{tj}$ and $\text{diff}_{tj} = Z_t - 1 - \text{rem}_{tj}$, all committed via Hyrax. At a random $r_{\text{norm}}\in\mathbb{F}^{t_{\text{bits}}+d_{\text{bits}}}$ one cubic multi-batched sumcheck merges $S\mathbf{N}-\text{rem}-Z\mathbf{Y}^{\text{att}}=0$ and $\lambda(Z-1-\text{rem}-\text{diff})=0$ under a Fiat–Shamir scalar λ. Range proofs on rem, diff (in the 64-bit bucket of §3.5) enforce both to be non-negative, which combined with the second identity gives $0\le\text{rem}<Z$ — the unique floor-division witness. A second sumcheck binds Z_t to the committed $\phi(\mathbf{Q}^{\text{n}}),\phi(\mathbf{K}^{\text{n}})$; a causal variant binds Z to a prefix sum.

Binding lookups to post-norm tensors. The Lasso queries for ϕ are evaluated on $\mathbf{Q}^{\text{n}},\mathbf{K}^{\text{n}}$, not the pre-norm $\mathbf{Q},\mathbf{K}$. The prover commits both $C_{\mathbf{Q}},C_{\mathbf{K}}$ (used by the QKV sumcheck) and $C_{\mathbf{Q}^{\text{n}}},C_{\mathbf{K}^{\text{n}}}$ (used as ϕ inputs); the global ϕ Lasso binds its query indices to the post-norm commitments via one shared Hyrax batch open, and the LN sub-proof for LN_Q,LN_K ties pre- and post-norm together.

3.2 Additive sub-table feature map

Rather than fix a kernel, π-Former parametrises ϕ as a sum of learnable sub-table lookups. With input bit-width B, decomposition depth $c \mid B$, and chunk width $m = B/c$, define the integer index $x_{\text{int}} = \text{clamp}(\lfloor x/s \rfloor, 0, 2^B - 1)$ and chunk extractor $\chi_i(x_{\text{int}}) = (x_{\text{int}} \gg im) \,\&\, (2^m - 1)$. Then

$$\phi(x) \;=\; \sum_{i=0}^{c-1} T_i\big[\chi_i(x_{\text{int}})\big], \tag{1}$$

where $T_0, \ldots, T_{c-1} \in \mathbb{R}^{2^m}$ are learnable sub-tables. They are initialised so $\sum_i T_i \approx \text{GELU}$ and trained end-to-end via the straight-through estimator [4]. For $B = 16, c = 2$ this replaces a 65,536-entry monolithic table with two 256-entry sub-tables, matching Lasso's structured-decomposability condition (§2). Because Lasso prover cost is independent of table values, training the entries of every T_i end-to-end imposes zero proving-cost penalty.

3.3 Ternary projections

Following BitNet b1.58 [11], every projection matrix ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O, \mathbf{W}_1, \mathbf{W}_2$, and the LM head) is constrained to entries in $\mathcal{T} = \{-1, 0, +1\}$ scaled by a learnable per-layer scalar $\alpha \in \mathbb{F}$, so the forward pass is $\mathbf{y} = \alpha(\mathbf{x}\mathbf{W}) + \mathbf{b}$ with $\mathbf{W} \in \mathcal{T}^{m \times n}$. Training uses the magnitude-thresholded $w \mapsto \text{sign}(w) \cdot [|w| > 0.7\overline{|w|}]$ map with the straight-through estimator; α is multiplied in after the matrix product so the in-circuit weight stays ternary.

In the SNARK, the contraction $\mathbf{Y}(r_t, r_d) - \mathbf{b}(r_d) = \alpha \sum_k \mathbf{X}(r_t, k)\mathbf{W}(k, r_d)$ is a degree-2 sumcheck that materialises only the cheap vector $\alpha \cdot \mathbf{W}(\cdot, r_d)$. The prover folds the equality polynomial against ternary entries with $+=$/$-=$/skip; no field multiplication appears in the contraction.

3.4 Constraint-fused LayerNorm

Let $\mathbf{x} \in \mathbb{Z}^d$ be a row, $\text{sum\_x} = \sum_j x_j$, $\text{sq\_sum\_x} = \sum_j x_j^2$, and $\sigma = \lfloor \sqrt{\text{var\_x}}/d \rfloor$. The integer variance is $\text{var\_x} = d(d\,\text{sq\_sum\_x} - \text{sum\_x}^2)$, which expands $\sum_j (dx_j - \text{sum\_x})^2$ without taking sum_x^2 outside the sumcheck. π-Former proves a LayerNorm without divisions or square roots in five sub-protocols:

1. **Mean** (degree-2 sumcheck for $\widetilde{\text{sum\_x}}$).
2. **Square-sum** (cubic sumcheck for $\widetilde{\text{sq\_sum\_x}}(r_t) = \sum_j x_{\text{col}}(j)^3$ with three copies of x_{col}, keeping the round polynomial cubic).
3. **Sigma residual** (cubic multi-batched sumcheck combining $\sum_i \widetilde{\text{eq}}\,\widetilde{\text{sum\_x}}(i)^2$ and $\sum_i \widetilde{\text{eq}}(d\widetilde{\sigma}(i))^2$ under a Fiat–Shamir scalar λ_{sig}; the verifier reconstructs var_x algebraically).
4. **Floor-sqrt** (range-check $\text{var\_x} - (d\sigma)^2 \geq 0$ and $(d\sigma + d)^2 - 1 - \text{var\_x} \geq 0$ via §3.5).
5. **Y fusion** (at one random (r_{y_t}, r_{y_d}), check $\gamma \odot (d\mathbf{x} - \text{sum\_x}) + \beta \cdot d\sigma \approx d\sigma \cdot \mathbf{y}$ via a range proof on the lo/hi residuals; the $\gamma\mathbf{X}$ and $\sigma\mathbf{Y}$ legs share one multi-batched cubic sumcheck).

Per LN the prover commits three internal Hyrax values $(\text{sum\_x}, \text{sq\_sum\_x}, \sigma)$; the squared quantities are bound algebraically by step 3 and need no separate commitment. The verifier evaluates γ, β from the VK in $O(d)$ per layer.

Model-level LN batching. After Phase 1 has absorbed all $5L+1$ LN IO commitments, one ProveLNsBatched call folds mean / variance / σ-floor / Y-fusion sub-claims of every LN into a single batched cubic sumcheck under Fiat–Shamir powers of an independent challenge. Soundness follows from the same Schwartz–Zippel argument as cross-block batching (Theorem 3) with L replaced by $5L+1$.

3.5 Globally batched range proof

The range proof shows $V[i] \in [0, 2^{\text{bits}})$ via chunked LogUp [6] over an identity table, with chunk width 16 and bucket widths $\{32, 64\}$. LayerNorm σ/y witnesses use the 64-bit bucket; each non-empty bucket produces one multiplicity commitment m_{com}.

For B witnesses in a bucket with $C = \lceil \text{bits}/16 \rceil$ chunks each, Phase 1 commits all $V_0^{(b)}, \ldots, V_{C-1}^{(b)}$ via Hyrax and merges chunk arrays into one global multiplicity vector $m[v] = \sum_{b,c,i} [\text{chunk}^{(b,c)}[i] = v]$ committed once. Phase 2 runs one degree-2 sumcheck per witness binding $V^{(b)}$ to a random $r_v^{(b)}$ and verifies $V^{(b)}(r_v^{(b)}) = \sum_c 2^{16c} V_c^{(b)}(r_v^{(b)})$ through one Hyrax batch open. Phase 3 opens $m(r_m)$ once and checks the LogUp identity against $T_{\text{id}}[i] = i$. The transcript-ordering invariant (Theorem 5) is that m_{com} is absorbed before any Phase 2 sumcheck challenge.

Zerocheck-folded inverse polynomial. The LogUp identity requires per-chunk inverse polynomials $h_c[i] = 1/(\alpha - V_c[i])$ for a Fiat–Shamir challenge α. A naïve implementation Hyrax-commits each h_c and opens it alongside the chunk commitments, costing $B \cdot C$ extra Pedersen MSMs per bucket. We avoid those commitments by folding a zerocheck of

$$h(x) \cdot (\alpha - V(x)) - 1 \;\equiv\; 0 \qquad (x \in \{0,1\}^n)$$

into the bucket's existing cubic sumcheck. Concretely, after the bucket claim $\Lambda = \sum_p w_p \cdot \big(\sum_i h_p[i] + \beta \cdot n_{\text{pad}}\big)$ is absorbed, the verifier samples $\gamma_{\text{fold}} \in \mathbb{F}$ and $r_{zc} \in \mathbb{F}^n$, and the prover proves

$$\sum_x \Big[w(x) \cdot h(x) \cdot \big(1 + \beta(\alpha - V(x))\big) + \gamma_{\text{fold}} \cdot \text{eq}(r_{zc}, x) \cdot \big(h(x)(\alpha - V(x)) - 1\big) \Big] =$$

The first term is the standard LogUp bucket claim; the second is the zerocheck term, which sums to zero on the boolean hypercube iff $h = 1/(\alpha - V)$ pointwise. The integrand remains cubic in (h, V, w, eq), so per-round prover cost is unchanged. We pre-fill h's pair-padding slots with α^{-1} so that the zerocheck term vanishes there ($\alpha^{-1} \cdot \alpha - 1 = 0$), keeping the bucket claim well-formed without an indicator mask.

Crucially, Λ is absorbed into the transcript *before* γ_{fold} and r_{zc} are sampled. The terminal $h(r_{\text{full}})$ is sent as a scalar (not opened against any commitment) and is bound only by the sumcheck's round-polynomial consistency together with the combined-identity check; the chunk evaluation $V(r_{\text{full}})$ remains bound to the Hyrax-committed V_c via the per-chunk openings at the bucket challenge r_k.

The model-level prover routes *all* range witnesses across the model into one bucketed list: $10L+2$ LN witnesses (σ, y for the five LNs per block, plus the final LN's two), and $2L$ further witnesses (rem, diff per block) when normalised attention is enabled. Sharing m_{com} per bucket saves $(B-1)\sqrt{2^{16}}$ G1 MSMs per bucket; the zerocheck fold removes a further $B \cdot C$ Hyrax commits and as many opens, with savings scaling with L rather than capped at the per-block level.

4 The π-Former Protocol

This section gives the protocol. Algorithm 1 describes preprocessing; Algorithm 2 the model-level prover; Algorithm 3 the per-block "Phase 1" that commits intermediate matrices and proves both LayerNorms; Algorithm 4 the cross-block batched sumcheck primitive; and Algorithm 6 the verifier. The LayerNorm sub-prover (Algorithm 5) and the globally batched range proof are described in §3.4 and §3.5.

4.1 Preprocessing

Algorithm 1 π-Former.$\mathsf{Setup}(\theta; \text{params})$

Input: model weights $\theta = \{\mathbf{W}_Q^\ell, \mathbf{W}_K^\ell, \mathbf{W}_V^\ell, \mathbf{W}_O^\ell, \mathbf{W}_1^\ell, \mathbf{W}_2^\ell, \gamma^\ell, \beta^\ell\}_{\ell=1}^L$, final-LN params, LM head $\mathbf{W}_h$; params $(T, d, d_{ff}, d_h, B, c)$.
Output: proving key pk, verifying key vk.

derive Hyrax generators $g_j \leftarrow \mathrm{SHA3}(\texttt{"piformer-hyrax-gen"} \| j) \cdot G$.
for each weight matrix $\mathbf{W} \in \theta \cup \{\mathbf{W}_h\}$ **do**
    $C_{\mathbf{W}} \leftarrow \mathsf{Com}(\mathbf{W})$ ▷ Hyrax commit to ternary entries
end for
for each lookup sub-table $T_0, \ldots, T_{c-1}$ **do**
    $C_{T_i} \leftarrow \mathsf{Com}(T_i)$
end for
$\mathsf{vk} \leftarrow \{C_{\mathbf{W}}\} \cup \{C_{T_i}\} \cup \{\gamma^\ell, \beta^\ell\} \cup \{g_j\}$
$\mathsf{pk} \leftarrow \mathsf{vk} \cup \{\text{raw ternary arrays, sub-tables}\}$
return $(\mathsf{pk}, \mathsf{vk})$

The verifying key contains *no* raw weights, only commitments. The proving key additionally carries the ternary $\{-1, 0, +1\}$ arrays and the lookup sub-tables so the prover can fold the equality polynomial against them in $O(Td)$ time without any field multiplications.

4.2 The model-level prover

Algorithm 2 π-Former.$\mathsf{Prove}(\mathsf{pk}, W, \mathsf{inst})$

Input: proving key pk, witness W, instance inst (Lasso queries for $\phi(\mathbf{Q}), \phi(\mathbf{K}), \phi(\mathbf{M})$ at every block).

Output: model proof π.

$\mathsf{FS.init}(\texttt{"piformer"})$; $C_{\mathbf{x}_{\text{in}}} \leftarrow \mathsf{Com}(\mathbf{x}_{\text{in}})$; $\mathsf{FS.absorb}(C_{\mathbf{x}_{\text{in}}})$

Phase 1: per-block commit + LN IO absorption

$C_{\text{cur}} \leftarrow C_{\mathbf{x}_{\text{in}}}$

for $\ell = 1, \ldots, L$ **do**

  $p_\ell \leftarrow \textsc{Phase1}(W_\ell, C_{\text{cur}}, \mathsf{pk}_\ell, \mathsf{FS})$ ▷ commits 5 LN IO matrices, ϕ inputs, optional attn-norm aux

  $C_{\mathbf{X}^\ell_{\text{mid}}} \leftarrow C_{\text{cur}} + p_\ell.C_{\text{LN}_{\text{aon}}.y}$ ▷ sandwich-norm residual

  $C_{\text{cur}} \leftarrow C_{\mathbf{X}^\ell_{\text{mid}}} + p_\ell.C_{\mathbf{O}_{\text{ffn}}}$

end for

Phase 2: global random point

$r_{td} = (r_t \| r_{\text{out}}) \leftarrow \mathsf{FS.challenge}(t_{\text{bits}} + d_{\text{bits}})$

Phase 3: four cross-block sumchecks

$\pi_{qkv} \leftarrow \textsc{CrossBatch}(\{f^{qkv}_\ell, g^{qkv}_\ell\}_\ell, \mathsf{FS})$

$\pi_{op} \leftarrow \textsc{CrossBatch}(\{f^{op}_\ell, g^{op}_\ell\}_\ell, \mathsf{FS})$

$\pi_{ao} \leftarrow \textsc{CrossBatch}(\{f^{ao}_\ell, g^{ao}_\ell\}_\ell, \mathsf{FS})$

$\pi_{ac} \leftarrow \textsc{CrossBatch}(\{f^{ac}_\ell, g^{ac}_\ell\}_\ell, \mathsf{FS})$

Phase 4: attention normalisation sumcheck (when normalised mode)

$r_{\text{norm}} \leftarrow \mathsf{FS.challenge}(t_{\text{bits}} + d_{\text{bits}})$; $\lambda \leftarrow \mathsf{FS.challenge}$

$\pi_{\text{atn}} \leftarrow \textsc{ProveCubicMulti}\big(\{\mathbf{N}_\ell, Z_\ell, \mathbf{Y}^{\text{att}}_\ell, \text{rem}_\ell, \text{diff}_\ell\}_\ell, \lambda, \mathsf{FS}\big)$

$\pi_Z \leftarrow \textsc{CrossBatch}(\{\phi(\mathbf{Q}^{\text{n}})_\ell, \sum_s \phi(\mathbf{K}^{\text{n}})_\ell\}_\ell, \mathsf{FS})$

Phase 5: global FFN Lasso + FFN-Y/M sumchecks

commit $\{\mathbf{A}_\ell, \mathbf{M}_\ell\}_\ell$; $\pi^{\text{ffn}}_{\text{lasso}} \leftarrow \mathsf{Prove\text{-}LassoMulti}\big(\{\mathsf{inst}_\ell.\phi(\mathbf{M})\}_\ell, \{(C_{\mathbf{A}_\ell}, \mathbf{A}_\ell)\}_\ell, \mathsf{FS}\big)$

bind FFN indices to $\{C_{\mathbf{M}_\ell}\}_\ell$ via one Hyrax batch open

$\pi_{ffy} \leftarrow \textsc{CrossBatch}(\{f^{ffy}_\ell, g^{ffy}_\ell\}_\ell, \mathsf{FS})$

$\pi_{ffm} \leftarrow \textsc{CrossBatch}(\{f^{ffm}_\ell, g^{ffm}_\ell\}_\ell, \mathsf{FS})$

Phase 6: bucketed range batch + model-level batched LN proof

for $\text{bits} \in \{32, 64\}$ **do**

  $\pi_{\text{rng,bits}} \leftarrow \textsc{ProveRangeBatched}(\text{all witnesses in bucket, bits, F}$

end for

$\pi_{\text{ln}} \leftarrow \textsc{ProveLNsBatched}(\text{all } 5L{+}1 \text{ LNs}, \mathsf{FS})$

Phase 7: LM head + μ-challenges + batch opens

$\pi_{\text{head}} \leftarrow \mathsf{Prove\text{-}Proj}(W.\text{LM-head}, \mathsf{FS})$

drain 13 deferred-accumulator μ-challenges plus 2 read-only ρ fuse challenges; finalise via `finalize_many_with_mus` (one MSM pair per Hyrax-params group)

$\pi_{\text{inter}} \leftarrow \mathsf{Open\text{-}Batch}\big(\{\mathbf{Q}_\ell, \mathbf{K}_\ell, \mathbf{V}_\ell, \mathbf{O}^\ell_{\text{att}}, \mathbf{O}^\ell_{\text{ffn}}\}_{\ell=1}^{L}, r_{td}\big)$

$\{\pi^{(j)}_{\text{open}}\} \leftarrow \mathsf{Open\text{-}Batch}(\{\mathbf{X}^\ell_{\text{n1}}, \mathbf{W}^\ell_Q, \ldots, C_{\phi(\mathbf{Q}^{\text{n}})_\ell}, C_{\phi(\mathbf{K}^{\text{n}})_\ell}\}_\ell, r^{(j)})$

(when normalised) $\mathsf{Open\text{-}Batch}([\mathbf{N} \mid \mathbf{Y}^{\text{att}} \mid \text{rem} \mid \text{diff}]_\ell, r_{\text{norm}})$

Phase 8: global $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$ Lasso

commit $\{\phi(\mathbf{Q}^{\text{n}})_\ell, \phi(\mathbf{K}^{\text{n}})_\ell\}_\ell$; bind indices to $\{C_{\mathbf{Q}^{\text{n}}_\ell}, C_{\mathbf{K}^{\text{n}}_\ell}\}$

$\pi^{\text{qk}}_{\text{lasso}} \leftarrow \mathsf{Prove\text{-}LassoMulti}\big(\{\mathsf{inst}_\ell.\phi(\mathbf{Q}^{\text{n}}), \mathsf{inst}_\ell.\phi(\mathbf{K}^{\text{n}})\}_\ell, \ldots, \mathsf{FS}\big)$

return π

The key structural choices are: *(i) one global random point r_{td} is drawn after all per-block commitments are absorbed*, so all six cross-block sumchecks can share it; *(ii) roughly two dozen cross-cutting Hyrax accumulators* (LayerNorm row / *td*, projection W/b, LM-head W/b, range σ/Y/*m*, intermediate, quantization-FFN / -QK / *-m*, attn-norm, five cross-block batch-open groups) are pushed by every claim and finalised once at the end of the *verifier*, with one MSM pair per group of shared Hyrax parameters (MSM-fused across same-params groups via `finalize_many_with_mus`); *(iii) residuals are verified by linearity of the Hyrax PCS:* $C_{\mathbf{X}_{\text{mid}}} = C_{\mathbf{X}_{\text{in}}} + C_{\mathbf{O}_{\text{att}}}$ is computed by the verifier from the existing commitments and requires no proof.

Quantization-proof binding. The Lasso protocol works over integer indices $\mathsf{idx}_j \in [0, 2^B)$, but the committed input tensors $C_{\mathbf{M}_\ell}, C_{\mathbf{Q}^{\text{n}}_\ell}, C_{\mathbf{K}^{\text{n}}_\ell}$ carry *signed* field-element values. π-Former bridges this gap with two quantization sub-proofs, $\pi^{\text{ffn}}_{\text{quant}}$ and $\pi^{\text{qk}}_{\text{quant}}$. Per layer the verifier knows the quantization parameters $(s_{\text{num}}, s_{\text{den}})$ from the VK (with s_{num} a power of two) and the centred zero-point $\mathsf{zp} = 2^{B-1}$. The prover commits a remainder tensor $\mathsf{rem}_j = r_j s_{\text{den}} + \lfloor s_{\text{num}}/2 \rfloor - s_{\text{num}}(\mathsf{idx}_j - \mathsf{zp})$ via Hyrax, adds rem to the global range batch with bit-width $\log_2 s_{\text{num}}$, and at a Fiat–Shamir random point $\mathbf{r}$ opens both $\widetilde{r}$ and $\widetilde{\mathsf{rem}}$ via batched Hyrax. The verifier evaluates the *public* index list $\widetilde{\mathsf{idx}}(\mathbf{r})$ directly and checks

$$\widetilde{r}(\mathbf{r})\, s_{\text{den}} + \tfrac{s_{\text{num}}}{2}\widetilde{1}(\mathbf{r}) \stackrel{?}{=} s_{\text{num}}\big(\widetilde{\mathsf{idx}}(\mathbf{r}) - \mathsf{zp}\,\widetilde{1}(\mathbf{r})\big) + \widetilde{\mathsf{rem}}(\mathbf{r}). \quad (2)$$

Combined with the remainder range proof, this argument pins every public idx_j to the unique integer $\lfloor (r_j s_{\text{den}} + s_{\text{num}}/2)/s_{\text{num}} \rfloor + \mathsf{zp}$ consistent with the committed raw value r_j. A second, redundant self-consistency check (the Open-Batch over `ffn_lasso_bind_open` / `qk_lasso_bind_open`) verifies that the proof's index-MLE opening vector matches what the verifier reconstructs from the public indices.

The two model-level Lasso families have a slight structural asymmetry. The attention $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$ proof is a single committed-output `LassoMultiProof` (`all_lasso_proof`) that absorbs every $C_{\phi(\mathbf{Q}^{\text{n}})_\ell}, C_{\phi(\mathbf{K}^{\text{n}})_\ell}$ before drawing batching challenges, then a second sumcheck inside the multi-proof binds the combined lookup grand sum to the committed outputs. The FFN family is rotated: a single cross-block π_{ffy} degree-2 sumcheck reduces all $\mathbf{A}_\ell$ leaves to one terminal claim $\sum_\ell w_\ell \widetilde{\mathbf{A}}_\ell(r_t, r_{k_{fy}}) = c$, and a separate *Lasso terminal-eval proof* (π^{term}_{ffn} = `ffn_a_terminal_proof`) proves c is the Lasso evaluation of the activation tables at the same terminal point. This fuses the equivalent of a separate $C_{\mathbf{A}_\ell}$ commitment, a Lasso multi-proof output binding, and a Hyrax batch open of $\{\mathbf{A}_\ell\}$ into one sub-protocol with no proof-carried $\mathbf{A}_\ell$.

The two model-level Lasso instances merit separate mention. Each is a *committed-output* Lasso: before sampling the multi-Lasso batching challenges, the prover absorbs the Hyrax commitments to the lookup outputs ($C_{\mathbf{A}_\ell}$ for FFN, $C_{\phi(\mathbf{Q}^{\text{n}})_\ell}$ and $C_{\phi(\mathbf{K}^{\text{n}})_\ell}$ for attention). A second sumcheck inside the Lasso multi-proof binds the combined lookup grand sum to the committed output polynomials at the sumcheck terminal point. Lookup indices are likewise bound to the committed *post-norm* input tensors ($C_{\mathbf{M}_\ell}, C_{\mathbf{Q}^{\text{n}}_\ell}, C_{\mathbf{K}^{\text{n}}_\ell}$) via one shared Hyrax batch opening, so a cheating prover cannot pick query indices independently of the committed inputs and cannot mix pre-norm and post-norm tensors.

4.3 Per-block Phase 1

Algorithm 3 PHASE1$(W_\ell, C_{\text{cur}}, \mathsf{pk}_\ell, \mathsf{FS})$

Input: block witness, current input commitment, block PK.
Output: intermediate commitments; *no* LN sumcheck or range proof emitted at this stage.

commit pre-/post-QK-norm $\mathbf{Q}, \mathbf{K}, \mathbf{V}$: $C_{\mathbf{Q}}, C_{\mathbf{K}}, C_{\mathbf{V}}, C_{\mathbf{Q}^{\text{n}}}, C_{\mathbf{K}^{\text{n}}}$
commit LN outputs: $C_{\mathbf{X}_{\text{n1}}}, C_{\text{LN}_{\text{aon}}.y}, C_{\mathbf{X}_{\text{n2}}}$
commit O-projection / FFN outputs: $C_{\mathbf{O}_{\text{att}}}, C_{\mathbf{O}_{\text{ffn}}}$
(when normalised attention) commit $C_{\mathbf{N}}, C_{\mathbf{Y}^{\text{att}}}, C_Z, C_{\text{rem}}, C_{\text{diff}}$
absorb LN_1 IO: $\mathsf{FS.absorb}(C_{\text{cur}}, C_{\mathbf{X}_{\text{n1}}})$
absorb $C_{\mathbf{Q}}, C_{\mathbf{K}}, C_{\mathbf{V}}$ and (when normalised) attn-norm aux commits
absorb LN_Q IO: $\mathsf{FS.absorb}(C_{\mathbf{Q}}, C_{\mathbf{Q}^{\text{n}}})$
absorb LN_K IO: $\mathsf{FS.absorb}(C_{\mathbf{K}}, C_{\mathbf{K}^{\text{n}}})$
$\mathsf{FS.absorb}(C_{\mathbf{O}_{\text{att}}})$
absorb LN_{aon} IO: $\mathsf{FS.absorb}(C_{\mathbf{O}_{\text{att}}}, C_{\text{LN}_{\text{aon}}.y})$
$C_{\mathbf{X}^\ell_{\text{mid}}} \leftarrow C_{\text{cur}} + C_{\text{LN}_{\text{aon}}.y}$ ▷ sandwich-norm residual 1
absorb LN_2 IO: $\mathsf{FS.absorb}(C_{\mathbf{X}^\ell_{\text{mid}}}, C_{\mathbf{X}_{\text{n2}}})$
$\mathsf{FS.absorb}(C_{\mathbf{O}_{\text{ffn}}})$
return all commitments

The only truly per-block step is Phase 1: it commits the per-block intermediate matrices and absorbs all five LN IO commitments (plus the optional attention-normalisation aux commits) into the transcript in a fixed order. *No per-block LN sumcheck and no per-block range proof are emitted at this stage*; both are deferred to model-level batched calls (Phase 6 of Algorithm 2). The remaining sub-protocols (QKV, output projection, attention, FFN) are *not* run independently per block either; they are batched cross-block in Phases 3–5 of Algorithm 2. Each per-block proof carries the intermediate Hyrax commitments, the FFN $C_{\mathbf{A}_\ell}, C_{\mathbf{M}_\ell}$, the per-block $C_{\phi(\mathbf{Q}^{\text{n}})_\ell}, C_{\phi(\mathbf{K}^{\text{n}})_\ell}$, the optional attention-normalisation aux commits, and the per-block scalar evaluations consumed by the cross-block batch sumchecks.

4.4 Cross-block batched sumcheck

Algorithm 4 CROSSBATCH$(\{f_\ell, g_\ell, H_\ell\}_{\ell=1}^{L}, \mathsf{FS})$

Input: per-block MLEs $f_\ell, g_\ell : \mathbb{F}^n \to \mathbb{F}$ and per-block targets H_ℓ.

Output: batched proof π_{batch} and shared random point $r \in \mathbb{F}^n$.

for $\ell = 1, \ldots, L$ **do** absorb H_ℓ and per-block constants into FS
end for
$\eta \leftarrow \mathsf{FS.challenge}(\texttt{"batch\_eta"})$
$H^\star \leftarrow \sum_\ell \eta^{\ell-1} H_\ell$; $P(x) \leftarrow \sum_\ell \eta^{\ell-1} f_\ell(x) g_\ell(x)$
for $i = 1, \ldots, n$ **do**
  send $s_i(X) = \sum_{x \in \{0,1\}^{n-i}} P_i(X, x)$ at $X \in \{0, 1, 2\}$
  $r_i \leftarrow \mathsf{FS.challenge}$
end for
return $(\eta, (s_i)_{i=1}^n), r = (r_1, \ldots, r_n)$

The verifier recovers $H^\star$ from the per-block claims and η, then runs the standard sumcheck verifier on π_{batch} to a final point r. From r the verifier checks the terminal claim $P(r) = \sum_\ell \eta^{\ell-1} f_\ell(r) g_\ell(r)$ against the per-block evaluations opened later via the cross-block batch opens. By Theorem 3 this protocol replaces L independent sumcheck transcripts with one of length n at a soundness cost of an additive $(L-1)/|\mathbb{F}|$.

4.5 Constraint-fused LayerNorm prover

The pseudocode below describes the LayerNorm protocol *for one LN witness*. The end-to-end prover does not invoke it once per LN; instead, it collects all $5L+1$ LN witnesses (in fixed order: per block $\mathrm{LN}_1, \mathrm{LN}_Q, \mathrm{LN}_K, \mathrm{LN}_{\text{aon}}, \mathrm{LN}_2$; then the final LN) and runs a single PROVELNSBATCHED call as Phase 6 of Algorithm 2. That batched call shares the row challenge r_t, the variance / Y-fusion sumchecks, and the deferred Hyrax accumulator pushes across all LN instances, with per-LN claims combined by Fiat–Shamir powers of an independent challenge.

Algorithm 5 PROVELN$(W, \{C_{\mathbf{x}}, C_{\mathbf{y}}\}, (\mathsf{rps}_\sigma, \mathsf{rps}_y), \mathsf{FS})$

Input: witness $W = \{\mathbf{x}, \mathbf{y}, \text{sum\_x}, \text{sq\_sum\_x}, \text{sum\_x\_sq}, \sigma, \sigma\text{\_sq\_scaled}\}$, where $\text{sum\_x\_sq} = \text{sum\_x}^2$ and $\sigma\text{\_sq\_scaled} = (d\sigma)^2$ are square-witness fields introduced so each algebraic identity collapses to a degree-≤ 3 multilinear sumcheck.

commit $C_{\text{sum\_x}}, C_{\text{sq\_sum\_x}}, C_\sigma$; absorb IO and internals into FS
$r_t \leftarrow \mathsf{FS.challenge}(t_{\text{bits}})$
// 1. Mean sumcheck (degree-2)
$(\pi^{\text{mean}}, r_d^{\text{m}}) \leftarrow \text{Prove-Sumcheck}_2(\widetilde{\mathbf{x}}_{\text{col}}, \mathbb{1}, \widetilde{\text{sum\_x}}(r_t), \mathsf{FS})$
// 2. Square-sum sumcheck (cubic): $\widetilde{\text{sq\_sum\_x}}(r_t) = \sum_j \widetilde{\mathbf{x}}_{\text{col}}(j)^3$ *via three copies of* $\widetilde{\mathbf{x}}_{\text{col}}$
$(\pi^{\text{sqs}}, r_d^{\text{v}}) \leftarrow \text{Prove-Sumcheck}_3(\widetilde{\mathbf{x}}_{\text{col}}, \widetilde{\mathbf{x}}_{\text{col}}, \widetilde{\mathbf{x}}_{\text{col}}, \widetilde{\text{sq\_sum\_x}}(r_t), \mathsf{FS})$
// 3. Sigma-residual sumcheck (cubic multi-batched, `sigma_residual_batch_lambda`*)*
$r_{\sigma_t} \leftarrow \mathsf{rps}_\sigma.r[..t_{\text{bits}}]$; $\lambda_{\text{sig}} \leftarrow \mathsf{FS.challenge}$
bind $\widetilde{\text{sum\_x\_sq}}(r_{\sigma_t}) = \sum_i \widetilde{\text{eq}}(r_{\sigma_t}, i) \widetilde{\text{sum\_x}}(i)^2$ and $\widetilde{\sigma\text{\_sq}}(r_{\sigma_t}) = \sum_i \widetilde{\text{eq}}(r_{\sigma_t}, i)(d\widetilde{\sigma}(i))^2$ in one cubic multi-batched sumcheck π^{sig}
// 4. Sigma range proof (consumes rps_σ*)*
recover $\text{var}(r_{\sigma_t}) = d\big(d\widetilde{\text{sq\_sum\_x}}(r_{\sigma_t}) - \widetilde{\text{sum\_x\_sq}}(r_{\sigma_t})\big)$; check $\text{var} - (d\sigma)^2 \geq 0$ and $(d\sigma + d)^2 - 1 - \text{var} \geq 0$ via rps_σ
// 5. Y constraint fusion: $\gamma\mathbf{X} + \sigma\mathbf{Y}$ *legs (multi-cubic)*
$r_y \leftarrow \mathsf{rps}_y.r$; $\alpha \leftarrow \mathsf{FS.challenge}(\texttt{"layernorm\_alpha"})$
$(\pi^{\gamma\sigma}, r_f) \leftarrow \text{Prove-Sumcheck}_3^{\text{multi}}\big([\gamma \odot \widetilde{\mathbf{x}}_{\text{ry}}, \widetilde{\sigma}_{\text{ry}} \odot \widetilde{\mathbf{y}}_{\text{ry}}], \alpha, \mathsf{FS}\big)$
push Hyrax claims into deferred accumulators $\mathsf{acc}_t, \mathsf{acc}_{td}$ (the model-level batched LN proof groups LNs by shape and folds all per-LN claims under one extra batching η)
return $\{\pi^{\text{mean}}, \pi^{\text{sqs}}, \pi^{\text{sig}}, \pi^{\gamma\sigma}, \text{evaluations}\}$

4.6 The verifier

Algorithm 6 π-Former. $\mathsf{Verify}(\mathsf{vk}, \pi, \mathsf{inst}, \mathbf{x}_{\text{in}}, \mathbf{y})$

check $\mathsf{Com}(\mathbf{x}_{\text{in}}) = \pi.C_{\mathbf{x}_{\text{in}}}$ ▷ public input is bound by re-commitment; the public output is bound below by evaluating $\widetilde{\mathbf{y}}$ at a transcript-derived random point and chaining through the LM-head proof
$\mathsf{FS.init}(\texttt{"piformer"})$; absorb $\pi.C_{\mathbf{x}_{\text{in}}}$; initialise the $\sim$20 deferred Hyrax accumulators (one per claim-class / Hyrax-params group)
$C_{\text{cur}} \leftarrow \pi.C_{\mathbf{x}_{\text{in}}}$
for $\ell = 1, \ldots, L$ **do** ▷ Phase 1 lockstep: absorb all 5 LN IO + aux
    absorb LN_1 IO; absorb $C_{\mathbf{Q}}, C_{\mathbf{K}}, C_{\mathbf{V}}$ + (opt.) attn-norm aux
    absorb LN_Q IO ($C_{\mathbf{Q}}, C_{\mathbf{Q}^{\text{n}}}$), LN_K IO ($C_{\mathbf{K}}, C_{\mathbf{K}^{\text{n}}}$), $C_{\mathbf{O}_{\text{att}}}$
    absorb LN_{aon} IO; $C_{\mathbf{X}_{\text{mid}}} \leftarrow C_{\text{cur}} + p_\ell.C_{\text{LN}_{\text{aon}}.y}$
    absorb LN_2 IO; $C_{\text{cur}} \leftarrow C_{\mathbf{X}_{\text{mid}}} + p_\ell.C_{\mathbf{O}_{\text{ffn}}}$
end for
$r_{td} \leftarrow \mathsf{FS.challenge}(t_{\text{bits}} + d_{\text{bits}})$
verify four cross-block sumchecks $\pi_{qkv}, \pi_{op}, \pi_{ao}, \pi_{ac}$
(when normalised) **verify** π_{atn} at r_{norm} and π_Z
verify global FFN Lasso, then π_{ffy}, π_{ffm}
verify bucketed range batch (one m_{com} per width)
verify model-level batched LN proof (all $5L+1$ LNs)
sample $r_{\text{lm}} \leftarrow \mathsf{FS.challenge}(t_{\text{bits}} + v_{\text{bits}})$; compute $v_{\text{lm}} \leftarrow \widetilde{\mathbf{y}}(r_{\text{lm}})$ from the public output
verify LM-head sumcheck under the claim $(r_{\text{lm}}, v_{\text{lm}})$; chain the resulting input-side claim into acc_{td} against $\pi.C_{\text{final\_ln\_out}}$
drain 13 deferred-accumulator μ-challenges plus 2 read-only ρ fuse challenges; finalise via `finalize_many_with_mus` (one MSM pair per Hyrax-params group)
verify global intermediate batch open at r_{td}
verify cross-block matrix-type batch opens
(when normalised) **verify** batch opens at r_{norm}
FINALISE(the $\sim$20 deferred accumulators) ▷ one MSM pair per group
verify global $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$ Lasso
return ACCEPT

5 Soundness

We sketch the soundness analysis; the full proofs follow standard Schwartz–Zippel and PCS-binding arguments.

Lemma 1 (Sumcheck soundness)**.** *For an MLE f of individual degree $\leq \delta$, a cheating prover can make the sumcheck for $H = \sum_{x \in \{0,1\}^n} f(x)$ accept a false claim $H' \neq H$ with probability at most $n\delta/|\mathbb{F}|$.*

Lemma 2 (Hyrax binding)**.** *Under the discrete-log assumption on $\mathbb{G}$, the Hyrax PCS is computationally binding: no PPT adversary can open a single commitment to two different evaluations at the same point with non-negligible advantage.*

Theorem 3 (Cross-block batching is sound)**.** *Let* CROSSBATCH *(Algorithm 4) reduce L per-block claims $\{H_\ell\}$ to one batched claim $H^\star = \sum_\ell \eta^{\ell-1} H_\ell$, and let the inner sumcheck be sound with error $\leq n\delta/|\mathbb{F}|$. Then* CROSSBATCH *is sound with total error*

$$\varepsilon_{\text{batch}} \;\leq\; \frac{L-1}{|\mathbb{F}|} + \frac{n\delta}{|\mathbb{F}|}.$$

Proof. The challenge η is drawn from FS *after* every per-block claim and per-block witness commitment is absorbed. The prover is therefore committed to $\{H_\ell\}$ before learning η. If any single claim is false, $\hat{H}_{\ell^\star} \neq H_{\ell^\star}$, then $P(\eta) = \sum_\ell \eta^{\ell-1}(\hat{H}_\ell - H_\ell)$ is a non-zero polynomial in η of degree $\leq L-1$. By Schwartz–Zippel, $P(\eta) = 0$ holds for at most $L-1$ values of η, so a uniformly random η catches the false claim with probability $\geq 1 - (L-1)/|\mathbb{F}|$. Conditional on the combined claim being honest, the inner sumcheck has soundness $n\delta/|\mathbb{F}|$. A union bound gives the stated error. □

Theorem 4 (Homomorphic residuals)**.** *Let $C_{\mathbf{A}} = \mathsf{Com}(\mathbf{A})$, $C_{\mathbf{B}} = \mathsf{Com}(\mathbf{B})$. The verifier-computed commitment $C_{\mathbf{A}} + C_{\mathbf{B}}$ binds $\mathbf{A} + \mathbf{B}$ unconditionally given Hyrax binding (Lemma 2).*

Proof. Hyrax commits row-by-row as MSMs over public generators $\mathbf{g}$. By bilinearity of MSM, $C_{\mathbf{A}} + C_{\mathbf{B}} = (\text{MSM}(\mathbf{g}, A_i) + \text{MSM}(\mathbf{g}, B_i))_i = (\text{MSM}(\mathbf{g}, A_i + B_i))_i = \mathsf{Com}(\mathbf{A}+\mathbf{B})$. Binding of $\mathbf{A}+\mathbf{B}$ then follows from binding of $\mathbf{A}$ and $\mathbf{B}$. □

Theorem 5 (Globally batched range proof is sound)**.** *Sharing one m_{com} per width bucket across all range witnesses in the bucket (LN σ/y across all blocks plus the final LN, plus the optional attention-normalisation residuals) is sound: a cheating prover cannot supply chunk values outside $[0, 2^{16})$ for any single witness. Moreover, the zerocheck-folded inverse polynomial binds h to $1/(\alpha - V)$ on the boolean hypercube without committing h, with soundness error $O(n/|\mathbb{F}|)$ per bucket where n is the bucket variable count.*

Proof. Each bucket is processed independently and the argument applies per bucket.

Chunk-value binding. The transcript-ordering invariant is that m_{com} is absorbed before any per-witness sumcheck challenge $r_v^{(b)}$ in the bucket is sampled. Since m aggregates all chunk occurrences across all witnesses and all $C = \lceil \text{bits}/16 \rceil$ chunks per witness, the prover must commit to m before seeing any sumcheck challenge. If the prover supplied a chunk value $v^* \geq 2^{16}$, the LogUp identity [6] would require $m[v^*] \cdot T_{\text{id}}[v^*]$ to balance, but T_{id} has no entry for v^*, contradicting the committed m (Lemma 2).

Inverse-polynomial binding (zerocheck fold). Let $\tilde{h}$ be a (possibly malicious) prover's choice of inverse-polynomial

MLE, V the (committed) chunk MLE, and define $\tilde{s}(x) := \tilde{h}(x)(\alpha - V(x)) - 1$. The bucket sumcheck proves

$$\sum_x \big[w(x)\tilde{h}(x)(1+\beta(\alpha - V(x))) + \gamma_{\text{fold}} \cdot \text{eq}(r_{zc}, x)\tilde{s}(x) \big] = \Lambda,$$

where Λ is computed from the prover-supplied per-chunk sums and absorbed into the transcript *before* γ_{fold} and r_{zc} are sampled. If $\tilde{h} \neq 1/(\alpha - V)$ on the boolean hypercube, then $\tilde{s}$ is non-zero on at least one boolean point, hence non-zero as a polynomial of total degree $\leq 2n$, and by Schwartz–Zippel $\Pr_{r_{zc}}[\tilde{s}(r_{zc}) = 0] \leq 2n/|\mathbb{F}|$. Conditioning on $\tilde{s}(r_{zc}) \neq 0$, the true integrand value is linear in γ_{fold} with non-zero slope, so the probability it equals Λ over a fresh γ_{fold} is at most $1/|\mathbb{F}|$. If the sum differs from Λ, standard sumcheck soundness rejects with probability $\geq 1 - 3n/|\mathbb{F}|$. Union-bounding yields $\leq 6n/|\mathbb{F}|$ soundness error per bucket. The pair-padding positions are filled with $\tilde{h} = \alpha^{-1}$, on which both the main term ($w = 0$) and the zerocheck term ($\alpha^{-1}\alpha - 1 = 0$) vanish, so padding contributes nothing to either side of the identity.

Once $\tilde{h}$ is forced to the unique inverse on the hypercube, the per-chunk claims $\sum_i \tilde{h}_p[i]$ recover the true LogUp LHS, and the model-level grand-sum check $\sum_p \text{claim}_p = \text{rhs\_claim}$ closes the argument against the multiplicity-table side. □

Theorem 6 (Model-level batched LayerNorm is sound)**.** *The single* PROVELNSBATCHED *call covering all* $5L+1$ *LNs in the model (per block:* $\text{LN}_1, \text{LN}_Q, \text{LN}_K, \text{LN}_{\text{aon}}, \text{LN}_2$*; plus the final LN) is as sound as* $5L+1$ *independent LN sub-proofs run with independent challenges.*

Proof. The batched LN proof folds the $5L+1$ per-LN claims via Fiat–Shamir powers of an independent batching challenge η, drawn after every per-LN IO commitment $(\mathbf{x}, \mathbf{y}, \text{sum\_x}, \text{sq\_sum}, \sigma)$ has been absorbed into the transcript and after the bucketed range batch has committed to all σ/y chunks. The same Schwartz–Zippel argument as Theorem 3 applies (with L replaced by $5L+1$): a single false LN claim makes the combined polynomial in η non-zero of degree $\leq 5L$, caught with probability $\geq 1 - 5L/|\mathbb{F}|$. The shared inner sumchecks (mean, variance, Y-fusion) inherit Lemma 1 soundness on the combined claim. □

Theorem 7 (Attention-normalisation sumcheck is sound)**.** *When normalised attention is enabled, the cubic multi-batched sumcheck for* $S \cdot \mathbf{N} - \text{rem} - Z \cdot \mathbf{Y}^{\text{att}} = 0$ *and* $\lambda(Z - 1 - \text{rem} - \text{diff}) = 0$*, combined with the range proofs on* rem, diff *(Theorem 5) and the auxiliary Z sumcheck, soundly enforces the floor-division identity* $\mathbf{Y}^{\text{att}}_{tj} = \lfloor S \cdot \mathbf{N}_{tj} / Z_t \rfloor$ *at every* (t, j).

Proof sketch. Range proofs constrain $\text{rem} \geq 0$ and $\text{diff} \geq 0$. The second identity then gives $\text{rem} \leq Z - 1$, i.e. $0 \leq \text{rem} < Z$. The first identity expresses $Z \cdot \mathbf{Y}^{\text{att}} = S \cdot \mathbf{N} - \text{rem}$ with the unique quotient given $0 \leq \text{rem} < Z$. Both identities are checked at one Fiat–Shamir random point of $t_{\text{bits}} + d_{\text{bits}}$ variables with cubic-sumcheck error and a λ-batching error of $O(1)/|\mathbb{F}|$. The auxiliary Z sumcheck binds Z_t to $\phi(\mathbf{q}^{\text{n}}_t) \cdot \sum_s \phi(\mathbf{k}^{\text{n}}_s)$ via the already-committed $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$ MLEs. Hyrax binding closes the loop. □

Theorem 8 (π-Former end-to-end soundness)**.** *For any PPT adversary* $\mathcal{A}$*, the probability that* $\text{Verify}(\text{vk}, \pi, \text{inst}, \mathbf{x}_{\text{in}}, \mathbf{y}) = \text{ACCEPT}$ *but* $\mathcal{M}_\theta(\mathbf{x}_{\text{in}}) \neq \mathbf{y}$ *is at most*

$$\varepsilon_{\text{sound}} \leq \frac{m_{\text{sc}}\, n_{\max}\, \delta_{\max} + m_{\text{batch}}(L-1) + m_{\text{open}}}{|\mathbb{F}|} + \varepsilon_{\text{DL}},$$

where m_{sc} *is the number of sumcheck invocations after batching,* $m_{\text{batch}} = 6$ *is the number of cross-block batches,* m_{open} *covers the random-linear-combination checks used in batched openings and lookups,* $n_{\max}$ *is the maximum number of sumcheck variables,* $\delta_{\max} \leq 3$ *is the maximum round-polynomial degree, and* ε_{DL} *is the DL advantage of* $\mathcal{A}$ *on* $\mathbb{G}$.

Proof sketch. By hybrid argument over the L blocks. (i) By Lemma 2, any forged Hyrax opening yields a DL solution. (ii) Block-level chaining is sound by Theorem 4, with the sandwich-norm residual $C_{\mathbf{X}_{\text{mid}}} = C_{\mathbf{x}_{\text{in}}} + C_{\text{LN}_{\text{aon}}.y}$ inheriting binding from $C_{\text{LN}_{\text{aon}}.y}$. (iii) Within each block, the bucketed global range batch over all witnesses is sound by Theorem 5; the model-level batched LN proof covering all $5L+1$ LNs is sound by Theorem 6; when normalised attention is enabled, the floor-division identity is sound by Theorem 7; the cross-block sumchecks are sound by Theorem 3; the committed-output Lasso is sound by Lemma 2 plus Lemma 1. The Lasso index-binding step ties the lookup queries to the post-QK-norm tensors $C_{\mathbf{Q}^{\text{n}}}, C_{\mathbf{K}^{\text{n}}}$, which are themselves bound to $C_{\mathbf{Q}}, C_{\mathbf{K}}$ through the LN-batched proof for LN_Q, LN_K. (iv) The final layer plus LM head reduce to one LN plus one projection. A union bound over all sub-protocols gives the bound. For typical parameters ($L \leq 12$, $n_{\max} \leq 64$, $\delta_{\max} \leq 3$) over BN254, the statistical term is below 2^{-240} and is dominated by the DL term. □

5.1 Zero knowledge

The current implementation is *not* zero-knowledge: weight commitments are public VK entries, and intermediate evaluation claims (e.g., $\mathbf{X}_{\text{n1}}(r_t, r_d)$) are revealed in the sumcheck transcripts. Standard techniques apply: blinding all witness polynomials with random masking terms before commitment, and using zero-knowledge variants of sumcheck and Hyrax. We discuss this further in §7.

6 Implementation

We implement π-Former in $\sim$19 k lines of Rust over BN254 plus a PyTorch training pipeline. The Rust prover/verifier and CLI are organised as in Table 1.

Table 1: π-Former Rust implementation, lines of code per module.

Module	LoC
`poly/` (dense MLE, equality, helpers)	∼0.6 k
`pcs/` (Hyrax PCS, batch accumulator)	∼1.3 k
`subprotocols/` (sumcheck ×3, GKR combine)	∼1.5 k
`lookup/` (Lasso multi, range batched, quant.)	∼3.4 k
`attention/` (LN, projection, LLA, ternary check)	∼4.6 k
`ffn/` (FFN sub-prover)	∼0.7 k
`cross_layer/` (cross-layer projection sumcheck)	∼0.6 k
`prover.rs`, `verifier.rs`, `setup.rs`	∼6.0 k
`bin/piformer/` (CLI, codecs, sample)	∼0.5 k
Total	∼19 k

PyTorch pipeline. The training pipeline implements `TernaryLinear` (with STE), `StructuredLookupActivation` (ϕ with sub-table forward plus STE backward), and `LinearAttentionLayer`. The integer forward pass used to construct the witness is bit-identical to the Rust circuit, so a witness exported from PyTorch is accepted by the Rust prover with no quantisation gap. The exporter also emits the Lasso instances ($\phi(\mathbf{Q}), \phi(\mathbf{K}), \phi(\mathbf{M})$ tables, query indices, and outputs) directly from the same integer arithmetic.

Single-head, causal supported, no ZK. The current Rust prover supports single-head attention with optional end-to-end causal masking (selected at setup; the prover swaps in three cubic-multi-batched sub-protocols whose third multiplicand absorbs the causal indicator polynomial) and no zero-knowledge masking layer. Multi-head splits the attention sumchecks into parallel batches with the same protocol structure, and ZK is an additional blinding layer over committed witnesses (§7).

7 Discussion and Limitations

What is and is not in the SNARK. The Rust prover proves the full normalised LLA output $\mathbf{Y}^{\text{att}} = \lfloor S \cdot \phi(\mathbf{Q}^{\text{n}})(\phi(\mathbf{K}^{\text{n}})^{\top}\mathbf{V})/Z \rfloor$ followed by the ternary output projection $\mathbf{O}_{\text{att}} = \mathbf{Y}^{\text{att}}\mathbf{W}_O + \mathbf{b}_O$ and the sandwich-norm $\text{LN}_{\text{aon}}(\mathbf{O}_{\text{att}})$ before the residual sum. The Python pipeline supports two attention modes: `normalized_fixed` (proven end-to-end by the Rust circuit, including the floor division) and `prover` (un-normalised numerator only, retained for legacy export); `normalized_float` is rejected at export time. *Causal masking is fully supported* end-to-end: when `causal=true` the prover swaps in cubic-multi-batched `batch_attn_out_causal` / `batch_attn_ctx_causal` / `attn_z_causal_sumcheck` sub-protocols that fold the causal indicator polynomial into the third multiplicand, so no separate prefix-context commitment is needed. The remaining limitations are multi-head attention (the prover currently rejects $n_{\text{heads}} \neq 1$ at setup) and zero-knowledge masking. Neither changes the protocol architecture: multi-head splits the attention sumchecks into parallel batches, and ZK is an additional blinding layer over committed witnesses.

Model-quality evaluation. Because π-Former changes the attention kernel and constrains weights to ternary values, a deployment-quality comparison must train matched baselines under the same parameter budget, context length, dataset, and quantisation regime, then report both task quality and proof cost. We leave this to future work and present π-Former here as evidence that the co-designed architecture is efficiently provable.

From prototype to deployment. Our reference implementation is CPU-only and single-threaded. GPU MSM and parallel sumcheck [12] would translate directly to π-Former: every dominant inner loop (cross-block batched sumcheck, Lasso multi-instance, batched Hyrax open) is data-parallel, and the global Hyrax accumulators are pure MSMs at the boundary. Streaming token generation is the natural fit for Nova-style folding [9]: the running context matrix $\mathbf{M} = \phi(\mathbf{K})^{\top}\mathbf{V}$ is the IVC accumulator, and one folding step per token yields constant amortised proving cost.

Beyond π-Former. The Rust codebase already contains two stand-alone sub-protocols that are not yet wired into the end-to-end prover. The first is an in-circuit ternary-weight check (`attention/ternary_check.rs`) via a 4-element Lasso table $[0, 1, r-1, 0]$; folding this into setup yields a verifying key that carries a binding proof $\mathbf{W} \in \mathcal{T}^{m \times n}$. The second is a single cubic sumcheck (`cross_layer/projection.rs`) that collapses L per-layer projection sumchecks into one of length $\log L + \log d_{\text{in}}$ by introducing a layer-index variable; this gives a further factor of L saving on the projection transcript. Together with an EVM-targeted Hyrax verifier and a zero-knowledge masking layer, these lay out the path from a research prototype to an on-chain verifiable inference service.

8 Related Work

Verifiable ML. zkCNN [10] gave the first GKR-based proof system for neural network inference, focused on convolutional networks. [1] extends ZK to proofs of *training* for deep networks. zkLLM [15] adapts a custom IOP to softmax LLM inference using `tlookup`-style structured lookups, with `zkAttn` as a tailored attention sub-protocol; zkGPT [12] uses a Plonky2 backend with constraint fusion across operations of a transformer block. Both target the standard softmax + GeLU + LayerNorm stack and pay a significant cost for every non-arithmetic operation. π-Former departs by co-designing the model and proof system: every operation is either a sumcheck

or a Lasso lookup by construction, and the model is allowed to recover expressivity through learnable lookup tables that are free under Lasso.

Sumcheck and lookup arguments. The sumcheck protocol underlies modern transparent SNARKs [13, 16]. Lasso provides a practical lookup argument with sub-linear prover cost on structured-decomposable tables [2, 5, 14], and is the natural fit for the additive sub-table structure of π-Former's ϕ. LogUp-style multiplicity arguments [6] back the globally batched range proof. Hyrax [18] gives a transparent, discrete-log-based polynomial commitment with $O(\sqrt{N})$ proofs.

Linear attention and ternary networks. Linear attention [7] replaces softmax with a feature-map kernel and recovers the $O(Td^2)$-vs-$O(T^2d)$ complexity trade-off; [19] projects the keys and values to a low-rank space; Reformer [8] uses LSH for sparsity. BitNet b1.58 [11] introduced ternary weights as a replacement for FP16 dense layers in LLMs, motivated by inference energy on edge devices; π-Former re-purposes the same primitive for SNARK proving cost.

9 Conclusion

We presented π-Former, a SNARK for verifiable transformer inference in which the model is made SNARK-friendly so the proof system has nothing expensive to absorb. Three architectural changes drive the design: softmax becomes a kernel attention with a learnable additive-lookup feature map, every projection becomes ternary so its sumcheck contraction has no field multiplications, and LayerNorm becomes an integer reformulation provable through sumcheck and chunked range proofs. The proof system batches per-block sub-protocols into six model-level sumchecks, two global Lasso proofs, and one global range proof per block, with all final Hyrax openings flowing through ten deferred batch accumulators. We implemented the system in ∼19 k lines of Rust over BN254 and gave formal cross-block batching soundness theorems. We leave a deployment-quality model evaluation and the engineering steps (multi-head, ZK masking, GPU MSM, on-chain verifier) to future work; we believe π-Former provides a credible foundation for production-grade verifiable transformer inference.

References

[1] Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. Zero-knowledge proofs of training for deep neural networks. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4316–4330, 2024.

[2] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: Snarks for virtual machines via lookups. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33. Springer, 2024.

[3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[4] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[5] Oleg Fomenko and Anton Levochko. Understanding lasso: A novel lookup argument protocol. *Cryptology ePrint Archive*, 2025.

[6] Ulrich Haböck. Improving logarithmic derivative lookups using gkr. *Cryptology ePrint Archive*, 2022.

[7] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

[8] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

[9] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*, pages 359–388. Springer, 2022.

[10] Tianyi Liu, Xiang Xie, and Yupeng Zhang. Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2968–2985, 2021.

[11] Shuming Ma, Hongyu Wang, Shaohan Huang, Xingxing Zhang, Ying Hu, Ting Song, Yan Xia, and Furu Wei. Bitnet b1. 58 2b4t technical report. *arXiv preprint arXiv:2504.12285*, 2025.

[12] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. {zkGPT}: An efficient non-interactive zero-knowledge proof framework for {LLM} inference. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2045–2063, 2025.

[13] Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In *Annual International Cryptology Conference*, pages 704–737. Springer, 2020.

[14] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with lasso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 180–209. Springer, 2024.

[15] Haochen Sun, Jason Li, and Hongyang Zhang. zkllm: Zero knowledge proofs for large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4405–4419, 2024.

[16] Justin Thaler. *Proofs, arguments, and zero-knowledge*, volume 4. Foundations and Trends in Privacy and Security, 2022.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[18] Riad S Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zksnarks without trusted setup. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 926–943. IEEE, 2018.

[19] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.